

기말 06-09장 핵심개념 빈출정리

내일 시험 직전용. 2023/2024 기말 출제 스타일 기준으로 빈출 개념을 특별 표시하고, 헛갈리는 지점을 이해 중심으로 정리했습니다.

0. 출제패턴 먼저 잡기

이 교수님 기말은 "개념을 아는지"보다 "그 개념을 코드 빈칸과 실행 결과에 정확히 적용하는지"를 묻는 쪽입니다.

빈출반복 출제된 형태

- CMP 직후 signed/unsigned flag 조건 쓰기
- C/C++ 조건문을 jump label 구조로 바꾸기
- MUL/DIV/IDIV 실행 후 레지스터, 오류 여부 판단
- swap을 C convention 또는 STDCALL로 작성/호출
- heap, stack, paging 설명 중 틀린 문장 고르기
- REAL4 bit pattern과 little endian 메모리 순서

전략점수 따는 순서

- 조건 점프 표와 MUL/DIV 표를 먼저 외운다.
- 코드 빈칸은 "반대 조건으로 탈출"인지 "참이면 이동"인지 label부터 본다.
- 나눗셈 문제는 먼저 DIV인지 IDIV인지 표시한다.
- 프로시저 문제는 [ebp+8], [ebp+12], ret/ret 8만 실수하지 않는다.

1. 06장 Conditional Processing

빈출2023: CMP signed/unsigned 설명, Boolean 문자열 변환, .WHILE 변환. 2024: JA/JB/JE/JG/JL/JNE flag 조건, JG/LOOPNE 실행 결과.

1-1. Boolean 명령어는 "결과 저장 여부"가 핵심

명령어	이해	flag	시험 포인트
AND	원하는 bit를 0으로 끄는 마스크	CF=0, OF=0, SF/ZF/PF 갱신	소문자->대문자: bit 5 clear
OR	원하는 bit를 1로 켜는 마스크	CF=0, OF=0, SF/ZF/PF 갱신	대문자->소문자: bit 5 set
XOR	1인 bit만 toggle	CF=0, OF=0, SF/ZF/PF 갱신	대소문자 서로 바꾸기
NOT	모든 bit 반전	flag 변경 없음	참거짓 선지 단골 함정
TEST	AND처럼 flag만 계산	결과 저장 없음, CF=0, OF=0	특정 bit 검사

CMP	dest - src처럼 flag만 계산	operand 변경 없음	조건 점프 직전 거의 항상 등장
-----	-----------------------	---------------	-------------------

```
; ASCII 대소문자 핵심: bit 5
and al, 11011111b ; 소문자 -> 대문자
or  al, 00100000b ; 대문자 -> 소문자
xor al, 00100000b ; 대문자 <-> 소문자 toggle
test al,00100000b ; bit 5가 1인지 검사, al은 안 바뀜
```

1-2. 조건 점프 표는 그대로 써야 한다

무조건 암기JA/JB는 unsigned, JG/JL은 signed. cmp left, right 다음 조건은 항상 left ? right 기준으로 읽습니다.

관계	unsigned jump	flag 조건	signed jump	flag 조건
>	JA	CF=0 AND ZF=0	JG	ZF=0 AND SF=OF
>=	JAE	CF=0	JGE	SF=OF
<	JB	CF=1	JL	SF!=OF
<=	JBE	CF=1 OR ZF=1	JLE	ZF=1 OR SF!=OF
==	JE/JZ	ZF=1	JE/JZ	ZF=1
!=	JNE/JNZ	ZF=0	JNE/JNZ	ZF=0

함정80h는 unsigned로 128이지만 signed byte로는 -128입니다. 그래서 같은 CMP라도 JA와 JG 결과가 달라질 수 있습니다.

1-3. C 조건문을 ASM으로 바꾸는 법

기출 빈칸은 label 흐름을 봐야 합니다. IF문은 보통 조건의 반대를 써서 false로 보냅니다.

```
; if (a <= b && c != d) x = 1; else x = 0; signed
mov eax, a
cmp eax, b
jg  L_FALSE ; a <= b의 반대는 a > b
mov eax, c
cmp eax, d
je  L_FALSE ; c != d의 반대는 c == d
mov x, 1
jmp L_END
L_FALSE:
mov x, 0
L_END:
```

```

; if (a < b || c >= d) x = 1; signed
mov eax, a
cmp eax, b
jl L_TRUE      ; OR는 하나라도 참이면 true
mov eax, c
cmp eax, d
jge L_TRUE
jmp L_END
L_TRUE:
mov x, 1
L_END:

```

1-4. LOOPNE/LOOPE

명령어	동작
LOOPE/LOOPZ	ECX를 1 감소시킨 뒤 ECX != 0이고 ZF=1이면 점프
LOOPNE/LOOPNZ	ECX를 1 감소시킨 뒤 ECX != 0이고 ZF=0이면 점프

주의LOOPNE 자체가 ZF를 새로 만들지 않습니다. 직전의 INC, CMP, TEST 등이 만든 ZF를 그대로 봅니다.

2. 07장 Integer Arithmetic

빈출2023: imul eax,72 shift 최적화, MUL/DIV 참거짓, CDQ 직접 구현, 산술식 변환. 2024: DIV 실행 결과, 구구단 빈칸, shift 곱셈, 나머지 식 변환.

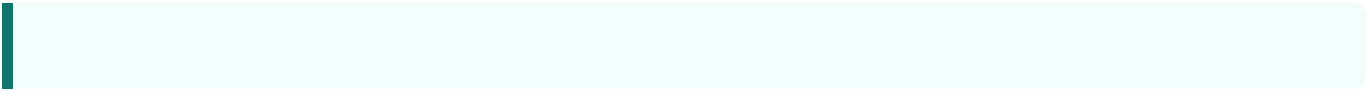
2-1. Shift와 Rotate

명령어	핵심 이해	시험 활용
SHL/SAL	왼쪽 shift, 빈 LSB는 0, 빠진 MSB는 CF	unsigned 곱하기 2^n
SHR	오른쪽 logical shift, 빈 MSB는 0	unsigned 나누기 2^n
SAR	오른쪽 arithmetic shift, sign bit 복제	signed 나누기 2^n
ROL/ROR	빠진 bit가 반대편으로 돌아옴	nibble 교환, bit 회전 추적

```

; var * 72 = var * (64 + 8)
mov eax, var
mov ebx, eax
shl eax, 6
shl ebx, 3
add eax, ebx

```



이해상수 곱셈 최적화는 2의 거듭제곱 합으로 분해합니다. 72는 $64+8$, 84는 $64+16+4$ 처럼 쪼갭니다.

2-2. MUL/IMUL 결과 위치

operand 크기	암시 피승수	결과	예
8비트	AL	AX	mul bl
16비트	AX	DX:AX	mul bx
32비트	EAX	EDX:EAX	mul ebx

- MUL은 unsigned, operand 1개, immediate 사용 불가.
- 결과 상위 절반이 0이 아니면 CF=1, OF=1.
- 1-operand IMUL은 signed이며 결과 위치는 MUL과 같다.
- 2/3-operand IMUL은 결과가 목적지 크기로 잘릴 수 있다.

2-3. DIV/IDIV는 표가 전부다

제수 크기	피제수	몫	나머지
8비트	AX	AL	AH
16비트	DX:AX	AX	DX
32비트	EDX:EAX	EAX	EDX

빈출DIV 전에는 상위 절반을 0으로 만든다. IDIV 전에는 부호 확장한다. 몫이 목적지 크기를 넘으면 divide error 입니다.

```
; unsigned 32-bit division
mov eax, dividend
mov edx, 0
div ebx      ; eax=quotient, edx=remainder

; signed 32-bit division
mov eax, dividend
cdq         ; eax sign을 edx 전체로 확장
idiv ebx    ; eax=quotient, edx=remainder
```

부호 확장	의미
CBW	AL -> AX
CWD	AX -> DX:AX
CDQ	EAX -> EDX:EAX

```

; VAR4 = (4 * VAR1) % (VAR2 - VAR3), signed DWORD
mov eax, VAR1
shl eax, 2
mov ebx, VAR2
sub ebx, VAR3
cdq
idiv ebx
mov VAR4, edx      ; 나머지는 EDX

```

2-4. ADC/SBB

큰 정수를 여러 조각으로 계산할 때 carry/borrow를 다음 조각으로 넘깁니다.

```

adc dest, src      ; dest = dest + src + CF
sbb dest, src      ; dest = dest - src - CF

```

3. 08장 Advanced Procedures

빈출2023: C calling convention의 swap 작성/호출. 2024: STDCALL의 swap 작성/호출. 두 해 모두 enter/leave 금지 조건이 붙었습니다.

3-1. Stack frame 위치

```

push ebp
mov  ebp, esp
sub  esp, local_size
...
mov  esp, ebp
pop  ebp
ret

```

위치	내용
[ebp]	이전 EBP
[ebp+4]	return address
[ebp+8]	첫 번째 parameter
[ebp+12]	두 번째 parameter
[ebp-4]	첫 번째 local DWORD

실수[ebp+4]는 첫 번째 인자가 아니라 return address입니다. 첫 번째 인자는 무조건 [ebp+8]부터 시작합니다.

3-2. C calling convention vs STDCALL

구분	C calling convention	STDCALL
push 순서	오른쪽 인자부터	오른쪽 인자부터
stack 정리	caller가 정리	callee가 정리
반환	ret 후 add esp,n	ret n
swap(&x,&y) 호출	push OFFSET ypush OFFSET xcall swapadd esp,8	push OFFSET ypush OFFSET xcall swap

3-3. swap 템플릿

```
; void swap(int *a, int *b)
swap PROC
    push ebp
    mov  ebp, esp
    sub  esp, 4      ; temp
    push eax
    push ebx
    push edx

    mov  eax, [ebp+8] ; eax = a
    mov  ebx, [ebp+12] ; ebx = b
    mov  edx, [eax]    ; edx = *a
    mov  [ebp-4], edx  ; temp = *a
    mov  edx, [ebx]    ; edx = *b
    mov  [eax], edx    ; *a = *b
    mov  edx, [ebp-4]
    mov  [ebx], edx    ; *b = temp

    pop  edx
    pop  ebx
    pop  eax
    mov  esp, ebp
    pop  ebp
    ret  8             ; STDCALL이면 ret 8, C convention이면 ret
swap ENDP
```

이해a, b는 int 값이 아니라 int의 주소입니다. 그래서 mov eax,[ebp+8]로 주소를 꺼내고, [eax]로 실제 int 값을 접근합니다.

3-4. 배열과 LEA

- 배열은 보통 값 전체가 아니라 시작 주소를 reference parameter로 전달한다.
- LEA는 메모리를 읽는 명령이 아니라 주소 계산 결과를 레지스터에 넣는다.

```
lea eax, [esi+ecx*4+12] ; eax = esi + ecx*4 + 12, 메모리 읽기 아님
```

4. 09장 Memory Management

빈출2023/2024 모두 heap, stack, logical/linear/physical address, page fault 설명의 옳고 그름을 물었습니다.
2024에는 REAL4 저장/해석도 출제되었습니다.

4-1. Stack, Data, Heap 구분

영역	무엇이 위치하나	시험 함정
Data segment	전역 변수, static 데이터	프로시저 호출과 무관하게 존재
Stack	return address, parameters, local variables, saved registers	지역변수는 return 과정에서 stack pointer 조정으로 사라짐
Heap	동적 할당 메모리	procedure return만으로 자동 해제되지 않음

오답 선지“동적 할당된 변수와 지역 변수는 모두 stack에 위치한다”는 틀립니다. 지역변수는 stack, 동적 할당 메모리는 heap입니다.

4-2. Win32 heap API

API	의미
GetProcessHeap	현재 프로세스의 기존 heap handle 반환
HeapCreate	새 private heap 생성
HeapAlloc	heap에서 memory block 할당, 포인터 반환
HeapFree	할당한 block 반환
HeapDestroy	HeapCreate로 만든 heap 제거

- HEAP_ZERO_MEMORY: 할당된 block을 0으로 초기화한다.
- HeapAlloc 실패 시 반환 포인터가 0인지 확인해야 한다.
- HeapFree 후 같은 포인터를 계속 쓰면 use-after-free 문제다.

4-3. Logical, Linear, Physical address

주소	뜻
Logical address	segment selector + offset
Linear address	segment descriptor의 base address + 32-bit offset
Physical address	실제 메모리 주소. paging이 켜져 있으면 linear address가 page translation을 거쳐 변환됨

순서 logical address -> segmentation -> linear address -> paging/page translation -> physical address

4-4. Paging과 page fault

- page는 보통 4KB 단위다.
- $4KB = 2^{12}$ 이므로 page offset은 하위 12비트다.
- 요청 page가 메모리에 없거나 접근 권한 문제가 있으면 page fault가 발생할 수 있다.
- cache miss는 page fault가 아니다.

linear address = 12345ABCh
4KB page offset = 하위 12비트 = ABCh

4-5. REAL4 IEEE 754

필드	크기	의미
sign	1비트	0은 양수, 1은 음수
exponent	8비트	bias 127
fraction	23비트	정규화된 1.x의 x 부분

$0.15625 = 0.00101b = 1.01b * 2^{-3}$
 sign = 0
 exponent = $-3 + 127 = 124 = 7Ch$
 fraction = 010000...
 bit pattern = 3E200000h
 x86 memory = 00 00 20 3E ; little endian

빈출REAL4 문제는 bit pattern을 구한 뒤 메모리에 적을 때 byte 순서를 뒤집는 것이 핵심입니다. 3E200000h는 낮은 주소부터 00 00 20 3E입니다.

5. 시험 직전 마지막 체크리스트

마지막 10분조건 점프 표, MUL/DIV 표, ret/ret 8, REAL4 little endian만 다시 보세요. 새 문제를 길게 풀기보다 실수 방지에 집중하는 쪽이 점수에 더 직접적입니다.

